CS6208 Progress Report

Ask or Act for Cooperative Assistance via Inverse Planning

Team member: Zhu Yufan A0238993J *March 13, 2026*

What Has Been Done

All milestones through March 13 are complete. We implement a cooperative 9×9 two-room gridworld where a principal holds a hidden goal object and issues a short, potentially ambiguous instruction that maps to $K \in \{1, 2, 3, 4\}$ candidate goals. The principal acts according to a Boltzmann policy with rationality β and noise ε [1, 3]; the assistant observes states and principal actions but not the true goal.

The assistant maintains a Bayesian posterior $b(g)$ over candidate goals, initialised from the instruction template and updated from observed actions via an inverse-planning likelihood. At each step it compares the expected team cost of acting immediately versus asking the best available clarifying question:

$$\text{CostAsk}(q) = 1 + c_q + \mathbb{E}_{a|q}[V(s, b_{q,a})],$$
$$\text{CostAct} = 1 + V(s, b, g^*),$$

where c_q=0.5 is the question cost, $b_{q,a}$ is the posterior after answer a, and V is the expected BFS distance to the MAP goal [2, 4]. This is a one-step value-of-information (VoI) decision rule.

Seven policies were implemented and evaluated: *AskOrAct* (VoI), *NeverAsk*, *AlwaysAsk*, *InfoGainAsk*, *EasyInfoGainAsk*, *RandomAsk*, and *POMCPPlanner* (a POMDP tree-search baseline). The question bank contains three question types: ask color, ask room, and ask object type, with per-type answer noise (0.05, 0.10, 0.20).

A full evaluation sweep was conducted over $\varepsilon \in \{0, 0.05, 0.1\}$, $\beta \in \{1, 2, 4\}$, 5 replicate seeds $\times$ 20 episodes per condition (100 episodes per cell, 19,800 total), with 95% bootstrap confidence intervals. Additional experiments include: ablations over question cost, wrong-pick failure mode, and episode deadline margin; robustness sweeps over answer noise and principal β-mismatch; held-out instruction template generalization; and a scale-K stress test with $K \in \{5, 6\}$.

Bug discovered and fixed. During evaluation, *AskOrAct* produced results identical to *NeverAsk* at K=2, asking zero questions across all episodes. The root cause was `ENTROPY_GATE = 0.8`: the policy skips the ask branch when $H(b) \leq$ `ENTROPY_GATE`, but the uniform prior over K=2 goals has entropy $\ln 2 \approx 0.693 < 0.8$, so the gate fired at every step. After reducing the threshold to 0.3 (below $\ln 2$ but above the near-certain regime), *AskOrAct* achieves 97% success vs. 85% for *NeverAsk* at K=2, ε=0, β=1. Three additional minor bugs were fixed: a NaN guard for unreachable goal positions, a degenerate-episode sentinel, and a warning for unregistered question IDs.

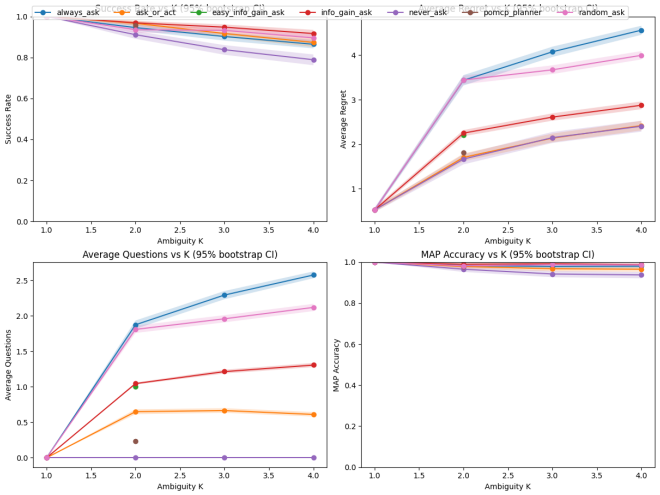

Figure 1: Main evaluation dashboard (100 ep/cell; error bars = 95% CI). Top: success rate. Middle: regret. Bottom: questions asked per episode.

Preliminary Findings

Figure 1 and Tables 1–2 summarise the key results. At K=1 (no ambiguity) all policies are equivalent — a sanity check confirming correct implementation. At $K \geq 3$ instruction ambiguity becomes genuine and policies diverge substantially.

AskOrAct outperforms *NeverAsk* with statistical significance at both high-ambiguity levels: +7.9 pp at K=3 (95% CI $[5.0, 10.8]$) and +8.4 pp at K=4 (95% CI $[5.1, 12.1]$), while asking only 0.6–0.7 questions per episode on average. Both confidence intervals exclude zero, confirming the effect is not due to sampling noise. *AlwaysAsk* asks 3–4$\times$ more per episode but achieves similar or lower success and substantially higher regret (+2.1 units at K=4), confirming that over-asking is costly.

Table 1: Results at K=3, ε=0, β=1 (100 episodes).

Policy	Success	Avg Q	Regret
AskOrAct	**90%**	0.74	**2.19**
NeverAsk	76%	0.00	2.51
InfoGainAsk	93%	1.25	2.58
AlwaysAsk	87%	2.41	4.14

Table 2: K=2, ε=0, β=1, post entropy-gate fix (100 ep.).

Policy	Success	Avg Q	Regret
AskOrAct	**97%**	0.71	**1.84**
NeverAsk	85%	0.00	2.16
InfoGainAsk	95%	1.04	2.44
AlwaysAsk	91%	1.97	3.67

Robustness (positive and negative findings). Answer-noise robustness sweeps show the *AskOrAct* advantage is retained at K=4 across all noise levels (+4–+9 pp), but the K=3 advan-

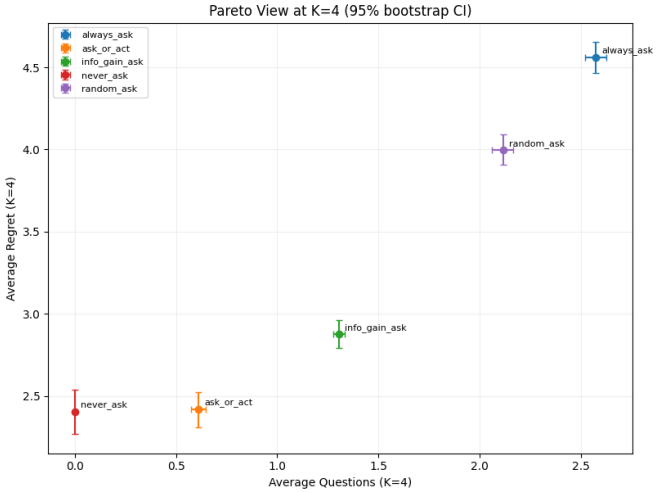

Figure 2: Success–regret Pareto frontier at $K{=}4$. *AskOrAct* is Pareto-efficient: it dominates *NeverAsk* in success and *AlwaysAsk*/*RandomAsk* in regret.

tage briefly reverses at noise $= 0.1$ (*AskOrAct* 92.0% vs. *NeverAsk* 94.0%) — a negative finding indicating that moderately noisy answers can mislead the posterior and cause unnecessary questions. Under principal model mismatch, *AskOrAct* retains a $+13.5$ pp advantage even when the principal acts near-randomly ($\beta_{\text{true}}{=}1$ vs. $\hat{\beta}{=}2$). The ablation confirms that a non-zero question cost is essential: setting $c_q{=}0$ causes the policy to over-ask (1.61 vs. 0.65 questions/ep) and reduces success from 73.8% to 69.2%.

Figure 2 shows the Pareto frontier at $K{=}4$. *AskOrAct* lies on the efficient frontier, dominating *NeverAsk* in success and *AlwaysAsk* and *RandomAsk* in regret, while *InfoGainAsk* achieves slightly higher success at the cost of more questions and higher regret.

Key Plot Description for Final Report

Figure 1 will appear as the central result figure. The horizontal axis is ambiguity level $K \in \{1, 2, 3, 4\}$; the three vertically stacked panels show (top) success rate, (middle) regret, and (bottom) average questions asked per episode, with one bar group per policy and 95% CI error bars. Readers learn: (a) all policies tie at $K{=}1$, validating the sanity baseline; (b) the success gap between *AskOrAct* and *NeverAsk* widens monotonically as ambiguity increases; (c) the contrast between panels (top) and (bottom) shows that *AlwaysAsk* and *RandomAsk* ask far more questions without a commensurate success gain. The three-panel layout allows readers to see the complete tradeoff in a single glance and directly supports the paper's central claim: selective asking captures most of the benefit of asking at a fraction of the cost.

Aggregate delta table (new in final report): rows are $K \in \{2, 3, 4\}$; columns are ΔSuccess (*AskOrAct*$-$*NeverAsk*) with 95% CI, and ΔRegret (*AskOrAct*$-$*AlwaysAsk*) with questions/ep. This table will let readers immediately see at which ambiguity levels the improvement is statistically significant and what it costs in terms of extra questions per episode.

Plan for Remaining Time

Step 1 — Failure-mode analysis (*by March 21*). Break down timeout failures versus wrong-pick failures per (K, ε, β) for all policies. Pinpoint conditions where *AskOrAct* regresses (notably $K{=}3$, noise $= 0.1$) and determine whether failures arise from posterior misdirection via noisy answers, an exhausted question budget, or an overly restrictive ask window. Findings will feed directly into the final report's discussion and limitations sections.

Step 2 — Final report (*by April 3*). Write the complete 4-page technical paper with introduction, related work, full method description (posterior update equations, VoI decision rule, gating conditions), experiments with aggregate tables and delta-CI analysis, ablation and robustness sections, discussion of limitations, and conclusion. All figures will be regenerated from the final corrected CSVs.

Step 3 — Presentation (*by April 8*). Prepare 14 slides covering problem setup, method, main results, Pareto frontier, robustness, ablations, and limitations, with backup slides addressing likely Q&A topics such as entropy gate sensitivity, question cost ablation, and scalability beyond $K{=}4$.

Step 4 — Submission package (*by April 10*). Run `./run.sh package` to build `submission_package.zip` containing all data CSVs, figure PNGs, report PDFs, and a `REPRODUCE.txt` with exact commands to regenerate every result from scratch, verifiable against expected row-count checksums for all seven output files.

AI Tool Declaration

I used Claude Sonnet 4.6 (Anthropic) for coding assistance, debugging, and text editing throughout the project. I am responsible for all research decisions, experimental design, and the content of this report.

References

[1] Chris L. Baker, Rebecca Saxe, and Joshua B. Tenenbaum. Action understanding as inverse planning. *Cognition*, 113 (3):329–349, 2009.

[2] Dylan Hadfield-Menell, Anca D. Dragan, Pieter Abbeel, and Stuart Russell. Cooperative inverse reinforcement learning. In *Advances in Neural Information Processing Systems*, 2016.

[3] Julian Jara-Ettinger, Hyowon Gweon, Laura E. Schulz, and

Joshua B. Tenenbaum. The naive utility calculus. *Trends in Cognitive Sciences*, 20(8):589–604, 2016.

[4] Zhi-Xuan Tan, Lexing Ying, Vikash Mansinghka, and Joshua B. Tenenbaum. Pragmatic instruction following and goal assistance via cooperative language guided inverse planning. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2024.